Draftroom

An Agentic Blog Writing Web Application

Web application design and implementation report

Abstract

Draftroom is a web application that turns a short writing brief into a blog article using a visible multi-stage agent workflow. A planner proposes an angle and outline, a writer creates a Markdown draft, a reviewer checks its quality, a reviser improves rejected work, and a final editor suggests titles and opening hooks. A chat assistant can update the brief and create or revise drafts from writing requests. The application was built with Express 5 on Node.js and browser JavaScript. It includes a clearly labeled sample mode and an AI mode that calls the OpenAI API from the server. The live website is https://draftroom-agent.onrender.com; the project source is https://github.com/yoclipit/draftroom-agent.

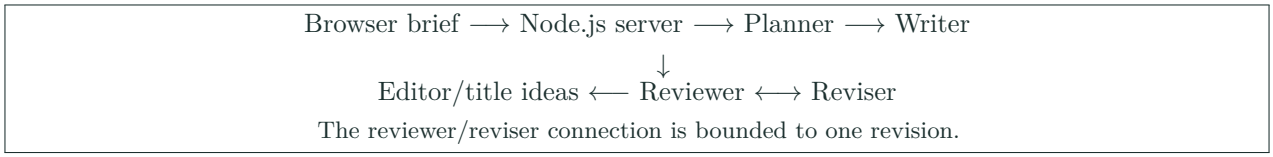
1 Problem and Goal

Starting with a topic alone often leads to an unfocused article. Draftroom asks for the intended audience, tone, goal, points to cover, and approximate length. The chat also accepts writing requests such as changing the tone, shortening a draft, or adding a requested point, and applies those changes to the brief. The user can inspect the plan and editorial notes, copy the Markdown, or download it for further editing.

The design follows a planner–writer–checker pattern. A quality loop needs an explicit stopping condition, so Draftroom ends its review cycle when the reviewer approves the article or after one revision and a final review.

2 System Architecture

The browser sends the brief to a Node.js HTTP server. The server validates the inputs, runs the agent stages in order, and streams progress events back to the page using Server-Sent Events. The chat sends the current brief and request to a separate API route. The server validates the requested action before updating the brief or running the writing workflow. In AI mode, only the server communicates with the model API. The API key is read from an environment variable and is never included in the browser response or repository.



Stage	Input	Output
Planner	Topic, audience, tone, goal, constraints	Working title, angle, section outline
Writer	Brief and outline	Markdown article
Reviewer	Brief and draft	Approval decision, score, strengths, fixes
Reviser	Rejected draft and fixes	Complete improved draft
Packager	Final article and brief	Three title ideas and two opening hooks

3 Implementation

The server uses Express 5 for routing, JSON parsing, and file delivery, with Node.js built-in modules for HTTP transport. The interface uses HTML, CSS, and browser JavaScript. It is responsive, supports keyboard navigation, and renders article text with DOM text nodes rather than injecting model output as HTML. The server accepts only the defined API routes and static files, limits request size, and applies a simple per-IP request limit.

Structured JSON responses are requested for planning, review, and title generation. The article itself is returned as Markdown. Each stage sends a progress event to the browser, which updates the activity list. The reviewer returns an approval decision and concrete changes. If the draft is rejected, the reviser produces a new complete article and the reviewer evaluates it once more. This gives the loop both a success condition and a hard upper bound.

Sample mode runs the visible sequence without contacting a model. Its output changes with all brief settings and is explicitly marked as a template preview. The no-key chat uses a limited command parser to change the brief, generate a sample draft, and start a copy or download. In AI mode, the model returns a structured plan containing only allowed writing actions. The server checks that plan and can revise an existing article before running the reviewer again. AI mode requires `OPENAI_API_KEY` in the server environment; no real key is stored in the project. The model can be changed with `OPENAI_MODEL`.

4 Verification and Limitations

Eight automated tests pass. They check brief validation, sample-stage events and field responsiveness, a mocked AI workflow that rejects and revises a draft, HTTP streaming and cache behavior, no-key chat commands, a chat HTTP request that creates a draft, and a mocked AI chat request that revises an existing article. The API key is absent from browser responses.

The live AI path was tested with a local mock service, not a personal API key. The public site has no AI key configured, so its chat uses the limited sample task mode. Model output may contain factual errors; the reviewer checks the writing brief and editorial quality but does not independently verify external claims. A person should check facts, citations, course policy, and final wording before publishing.

5 Artifacts and Access

- **GitHub repository:** `https://github.com/yoclipit/draftroom-agent`
- **Application source and tests:** linked in the repository above.
- **Local application:** run `npm start` and open `http://localhost:3000`.
- **Live application:** `https://draftroom-agent.onrender.com`
- **Deployment configuration:** a Render blueprint is included as `render.yaml`.

References

1. OpenAI, "Create chat completion," `https://developers.openai.com/api/reference/resources/chat/subresources/completions/methods/create`.
2. OpenAI, "Structured model outputs," `https://developers.openai.com/api/docs/guides/structured-outputs`.